

Control-Plane Architecture

the quantum-system Module and Deployment Modes

Version 1.4.2-SNAPSHOT, 2026-09-21T22:14:39Z

Table of Contents

1. Why a split	2
2. Module layering	3
3. The contracts	4
4. quantum.mode: what actually changes	5
5. The admin REST surface	6
6. Deployment tiers (WP3)	7
7. Identity in the split	8
8. Field-level and relationship policy (one policy source, every egress)	9
9. Commit trail (the integration branch)	10

This is the deep dive on the control-plane/data-plane split introduced on branch the `integration` branch (design source: [docs/design/CONTROL_PLANE_SPLIT_DESIGN.md](#); work plan: WP1 workstream B; compatibility contract: WP3). It explains what the `quantum-system` module is, why each contract lives where it does, what `quantum.mode` changes at runtime, and how the deployment tiers map to configuration.

Chapter 1. Why a split

Historically every Quantum app owned a **system realm** (`system-psa-com`, `system-b2bi-com`, ...) holding credentials, the realm catalog, and tenant provisioning state, and exposed the admin REST surface that manages them. A multi-tenant platform deployment needs those concerns in ONE control plane — tenant lifecycle, identity, catalog, metering — while existing apps keep working unchanged. The split is therefore **strictly opt-in**: every existing app remains a valid embedded deployment forever.

Chapter 2. Module layering

quantum-models	contracts & entities (Rule, Realm, UserRealmRole, FieldPolicyEnforcer, RoleSource, ...)
┆	
quantum-morphia-repos	repos + RuleContext (policy engine) + MigrationService
┆	
quantum-system	CONTROL-PLANE module: SystemDirectory + local impl, RealmCatalogService, TenantLifecycle contract, QuantumModeConfig / SystemRealmOwnership, RealmMembershipService, metering contracts.
┆	No JAX-RS resources in this jar.
┆	
quantum-framework	app framework: REST resources, seeds/bootstrap, startup coordination, TenantProvisioningService

Placement rules that shaped this:

- **Contracts sit as low as their consumers.** `MigrationService` lives in `quantum-morphia-repos` and therefore cannot see `quantum-system`; instead of making it mode-aware it gained an explicit `initializeStartupRealms(boolean includeSystemRealm)` overload — mode is resolved above it and passed down.
- **quantum-framework depends on quantum-system** (not vice versa), so every existing app's classpath is unchanged and the control-plane contracts are available to all of them.
- **The embedded implementations are the defaults.** `TenantLifecycle` is implemented by `TenantProvisioningService` by delegation — its existing signatures are untouched (WP3 rule: the contract mirrors, never replaces).

Chapter 3. The contracts

Contract	Owns	Embedded implementation
<code>SystemDirectory</code>	system realm id, credential lookup, realm resolution	<code>LocalSystemDirectory</code> over the configured system realm
<code>RealmCatalogService</code>	realm catalog reads/writes (the tenant → realm registry)	system-realm repos
<code>TenantLifecycle</code>	provision/delete tenants (<code>TenantProvisionRequest/Result</code>)	<code>TenantProvisioningService</code> (delegation)
<code>RealmMembershipService</code>	org/account membership per realm (one OWNER, N PARTICIPANTS) and per-realm user roles (<code>UserRealmRole</code>)	system-realm repos; consumed by the IdP at token issuance

In remote mode (Phase C, in progress) each contract gains an HTTP client implementation against the control-plane service; consumers are unaffected because they inject the contract.

Chapter 4. quantum.mode: what actually changes

`QuantumModeConfig` is the single source of the deployment mode; `SystemRealmOwnership` answers "does this deployment own the system realm?"

Startup concern	embedded	remote
System-realm migrations	runs	skipped (control plane owns them)
Baseline identity (system users)	runs	skipped
System-realm seed packs	runs	excluded from realm list
System-realm bootstrap packs	runs	excluded from realm list
App-realm migrations/seeds/bootstrap	runs	still runs locally
SystemDirectory	local	remote client (fails loud until Phase C)

Known Phase C gap (documented in the design doc): when seeding **app** realms, the seed runner still reads realm records and admin credentials from the local system-realm database; in a true split deployment those reads must route through `SystemDirectory`.

Chapter 5. The admin REST surface

The physical `quantum-system-rest` jar split landed in `1.4.1-SNAPSHOT`. Tenant applications omit that server module and therefore do not expose its security and control-plane endpoints. `quantum-auth-service` explicitly adds the module and publishes the API consumed through its generated SDK.

For deployments that include the server module, `quantum.system-rest.enabled` remains a build-time, default-true gate for the annotated control-plane admin resources. The next modularization step moves the resources' service closure (provisioning → seed subsystem and shared BaseResource REST infrastructure) below the server jar so it no longer needs the full `quantum-framework` dependency.

Chapter 6. Deployment tiers (WP3)

Tier	Config	Meaning
0 (default)	nothing	Today's behavior: app owns its system realm and admin surface. Expected end state for existing apps indefinitely.
1 (hardened)	<code>quantum.system-rest.enabled=false</code> at build	Same embedded behavior; the app stops exposing tenant provisioning/admin endpoints publicly.
2 (split planes)	<code>quantum.mode=remote</code> <code>quantum.system-service.base-url=...</code> tier-1 build switch	Control plane owns the system realm and identity; this app runs only app-realm workloads and validates JWTs against the control plane's JWKS. the tenant-plane app is the reference deployment.

Chapter 7. Identity in the split

Per the realm-membership ADR: users hold ONE global identity (credential in the system realm) plus per-realm role assignments (`UserRealmRole`). At token issuance the IdP unions the assignment for the target realm into the JWT's role claims (`CustomTokenAuthProvider.login`), with role provenance exposed in the login response (`RoleSource.REALM`). The tenant plane validates via JWKS and routes by claims — no per-request callback to the control plane.

Chapter 8. Field-level and relationship policy (one policy source, every egress)

The same rules that scope rows (`andFilterString/orFilterString`, including ontology traversals like `hasEdge('assignedTo', ...)`) also scope fields (`excludedFields`). Resolution happens once (`RuleContext.getExcludedFieldPaths`); enforcement is layered per egress: Mongo projection on reads, preserve-hidden-fields on merge, `FieldPolicyEnforcer.mask` for non-Mongo datasources, and the REST serialization interceptor as the last lock. See the Field-Level Policy section of [permissions.adoc](#).

Chapter 9. Commit trail (the integration branch)

[b8361878](#) SystemDirectory indirection · [854ea4f6](#) quantum-system module · [a2a78bb7](#) catalog routing · [851bdebe](#) quantum.mode seam · [edf71e78](#) system-rest switch · [28714ade](#) TenantLifecycle contract · [40c48926](#) field-level policy · [cf1a1641/0053f835/78f2b130/dcacf1a6/f91c85fe](#) membership model, ADR, docs, integration test, per-realm token claims.